

Job Hunter — Technical Report & Troubleshooting Guide

Version: 1.0 **Date:** June 2026 **Purpose:** Complete technical reference for the Job Hunter automated application system.

Part 1: High-Level Overview

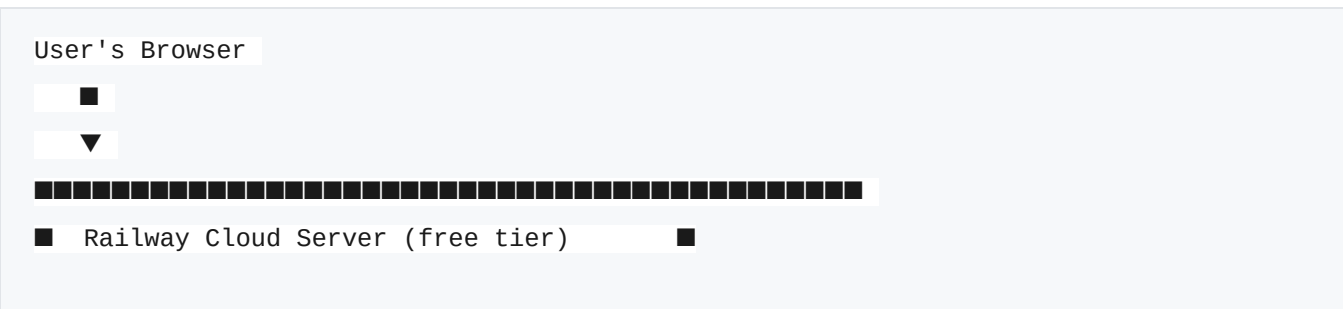
What This System Does

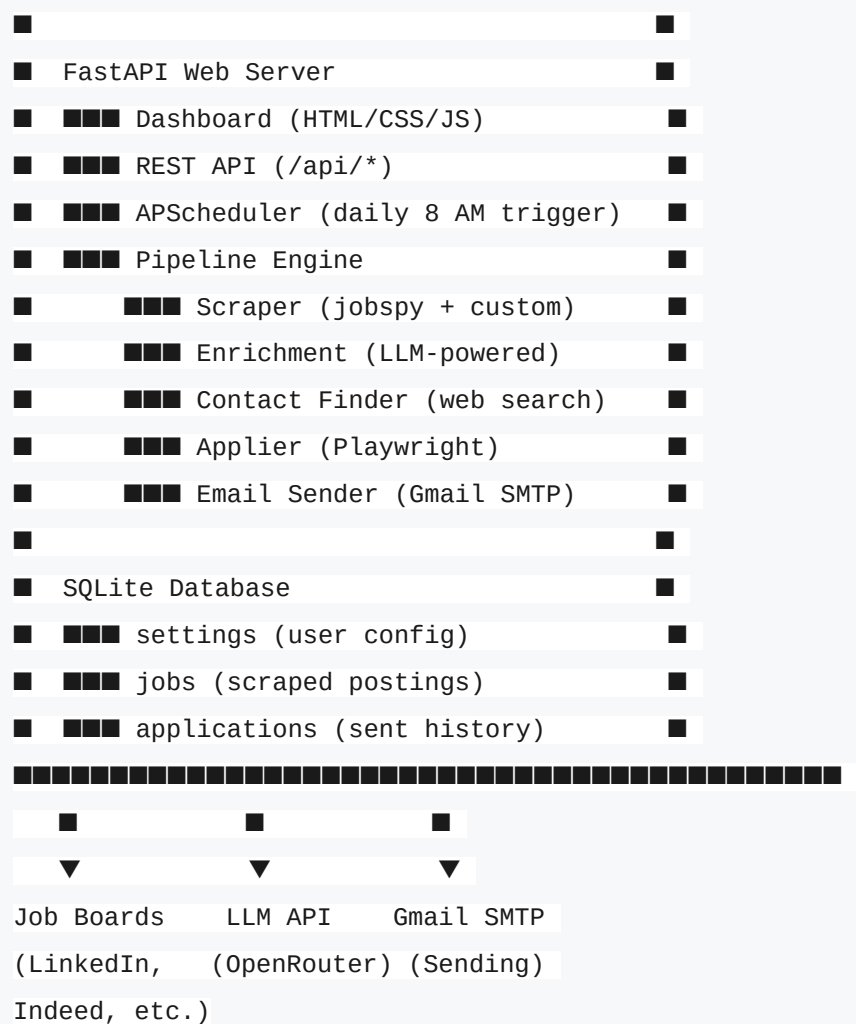
Job Hunter is a fully automated web application that applies to project management jobs on behalf of a job seeker. Every day at 8:00 AM (configurable), it:

1. **Scrapes** 7 job platforms for fresh postings (last 24 hours)
2. **Selects** the top 10 by relevance scoring
3. **Enriches** the resume and cover letter for each specific job using AI
4. **Finds** the hiring manager's email through free web research
5. **Applies** directly through each job platform using browser automation
6. **Emails** the hiring manager directly as a secondary outreach channel
7. **Logs** everything to prevent duplicate applications

The system runs on a cloud server (Railway free tier). The user interacts with it through a web dashboard — no command line, no code editing, no terminal.

Architecture Summary





Technology Stack

Component	Technology	Why We Chose It	Cost
Web Framework	FastAPI (Python)	Async, auto-generates API docs, lightweight	Free
Scheduler	APScheduler	Runs inside the Python process — no OS cron needed	Free

Database	SQLite	Zero config, single file, perfect for 10 writes/day	Free
Job Scraping	python-jobsby	Multi-site scraper (LinkedIn, Indeed, Glassdoor, Google, ZipRecruiter)	Free
Craigslist	Custom HTML scraper	jobsby doesn't support Craigslist; we parse the HTML directly	Free
Contact Finding	DuckDuckGo + DNS + SMTP	Web search for names, email permutation, mail server verification	Free
AI Enrichment	OpenRouter API	OpenAI-compatible; supports GPT-4o-mini, Claude, Llama, etc.	~\$0.01/app
Browser Automation	Playwright (Python)	Logs into platforms, fills forms, submits applications	Free
Email Sending	Gmail SMTP	Free, 500/day limit (we send max 10)	Free
Frontend	Vanilla HTML/CSS/JS	No build step, no npm, no framework — single file	Free

Hosting	Railway	Free tier, always-on, auto-deploy from GitHub	Free
Font	Inter (Google Fonts)	Clean, professional, used by Linear/Vercel/Stripe	Free

Part 2: Feature Breakdown

Feature 1: Multi-Platform Job Scraping

What It Does

Searches 7 job platforms simultaneously for postings matching the user's target roles and locations.

Supported Platforms

Platform	Method	Notes
LinkedIn	jobspy (built-in)	Largest professional network
Indeed	jobspy (built-in)	Largest job board globally
Glassdoor	jobspy (built-in)	Includes salary data
Google Jobs	jobspy (built-in)	Aggregates from many sources
ZipRecruiter	jobspy (built-in)	Strong in US market
Bing Jobs	jobspy (built-in)	Alternative aggregator

Craigslist	Custom scraper	HTML parsing — no API available
------------	----------------	---------------------------------

Region Support

The user selects a region (US, UK, Canada, Australia, Germany, etc.). This maps to the `country_indeed` parameter in jobspy, which filters results by country.

How Scraping Works

1. For each (job_title × location) combination, call jobspy with the selected platforms
2. jobspy returns a DataFrame with: title, company, location, description, job_url, date_posted, salary
3. For Craigslist, we send HTTP requests to `{city}.craigslist.org/search/jjj?query={title}` and parse the HTML
4. All results are deduplicated by (title, company) pair
5. Each job is scored 0-100 by relevance

Scoring Algorithm (0-100)

- **Title match** (0-40): Exact match to target titles = 40, partial word match = 25
- **Description quality** (0-15): >200 chars = 15, >50 chars = 8
- **Salary info** (0-15): Has salary = 15, below minimum = -30 penalty
- **Has apply URL** (0-10): Valid URL = 10
- **Has company name** (0-10): Non-empty = 10
- **Recency** (0-10): <6 hours old = 10, <12h = 7, <24h = 5

Troubleshooting

Problem	Cause	Fix
No jobs scraped	jobspy not installed	<code>pip install python-jobspy</code>

No jobs scraped	Too specific search terms	Broaden titles/locations in settings
Craigslist returns nothing	City name doesn't match CL subdomain	Check the city mapping in scrape.py
Rate limited by Indeed	Too many requests too fast	Reduce number of search titles
Jobs from wrong country	Region not set	Check <code>search.region</code> in settings

Feature 2: AI-Powered Resume Enrichment

What It Does

For each of the 10 selected jobs, the system uses an LLM to:

1. Analyze the job description (extract required skills, keywords, ATS terms)
2. Compare to the user's base resume
3. Generate a tailored resume and cover letter
4. Apply a unique style variant so no two applications look identical

Resume Modes

Honest Mode

- Reorders resume bullets to match job priorities
- Mirrors keywords from the job description
- Strengthens action verbs (managed → led, helped → delivered)
- Quantifies accomplishments
- Adjusts professional summary per role
- **Does NOT add experience the user doesn't have**

Enhanced Mode

- Everything from honest mode, PLUS:
- Adds relevant skills the job requires
- Embellishes accomplishments with plausible metrics
- Reframes existing experience to match the job's language
- Fills experience gaps with transferable skills
- **Still realistic — no obvious fabrications**

Style Variants (10 Total)

Each daily application gets a different variant to ensure variety:

#	Variant	Resume Angle	Cover Letter Hook
1	Achievement-focused	Lead with biggest measurable result	Open with a specific accomplishment
2	Skills-forward	Lead with technical skills	Open with a matching skill
3	Leadership-angle	Lead with team leadership	Open with a leadership story
4	Problem-solver	Problem → action → result bullets	Open with a challenge you solved
5	Results-driven	Focus on ROI and business impact	Open with the impact you'd bring
6	Collaborative	Focus on cross-functional work	Open with how you'd collaborate
7	Innovation-focused	Focus on process improvements	Open with an improvement idea

8	Client-centric	Focus on client/customer impact	Open with a client success story
9	Strategic	Focus on planning and vision	Open with your strategic approach
10	Delivery-focused	Focus on on-time delivery	Open with a delivery story

Context-Driven Customization

Every LLM call includes full context:

- **Job context:** Title, company, description, required skills, nice-to-haves, seniority level, company culture, industry, ATS keywords
- **Candidate context:** Name, email, phone, location, LinkedIn, base resume
- **Mode context:** Honest or enhanced, style variant number

ATS Optimization

- Standard section headers (Professional Summary, Core Competencies, Experience, Education)
- Clean markdown format (no tables, no columns, no graphics)
- Keyword density matching the job description
- One page equivalent length
- Strong action verbs, quantified results

How the LLM Call Works

1. System prompt sets the role (expert resume writer + ATS specialist)
2. User prompt includes all job + candidate context + mode instructions
3. LLM generates the tailored resume in markdown
4. If LLM fails, falls back to the base resume (still sends something)

Troubleshooting

Problem	Cause	Fix
LLM returns None	Invalid API key	Check <code>llm.api_key</code> in settings
LLM returns None	Wrong model name	Check <code>llm.model</code> (e.g., <code>openai/gpt-4o-mini</code>)
LLM returns None	Rate limited	System auto-retries after 15s; if persistent, wait
Resume sounds generic	JD too short	Short JDs get less specific customization
Applications look identical	Same style variant	Each application gets a different variant automatically
Enhanced resume has errors	LLM hallucination	This is expected — user should review enhanced resumes

Feature 3: Free Contact Finding

What It Does

Finds the hiring manager's email for each job without any paid API.

5-Layer Fallback Chain

1. **Extract from job description** — regex scan for email patterns
2. **Extract company domain** — parse the job URL, skip job board domains
3. **Web search for names** — DuckDuckGo search for "[company] hiring manager linkedin"
4. **Email permutation + SMTP verify** — generate common patterns (first.last@, f.last@, etc.), check if the mail server accepts them

5. **Generic emails** — try hr@, careers@, recruiting@, jobs@, talent@

SMTP Verification Technique

This is the key trick — we check if an email exists WITHOUT sending any email:

```
# Get the mail server for the domain
mx_records = dns.resolver.resolve(domain, "MX")
mx_host = str(sorted(mx_records)[0].exchange).rstrip(".")

# Connect and check (NO email sent)
server = smtplib.SMTP(mx_host, 25, timeout=5)
server.ehlo("verify.local")
server.mail("check@verify.local")
code, message = server.rcpt("john.smith@company.com")
server.quit()

# code == 250 means the email exists
# code == 550 means it doesn't
```

Troubleshooting

Problem	Cause	Fix
All contacts are "fallback"	Company domain not found	Normal for some jobs — fallback is used
SMTP verification times out	Mail server blocks verification	Falls back to generic email
DuckDuckGo returns nothing	Company name too generic	Falls back to domain-based guessing
Wrong email found	Common name collision	Contact source logged — check applications table

Feature 4: Platform Apply (Browser Automation)

What It Does

Logs into job platforms with the user's pre-created credentials and submits applications through the platform's own apply flow.

Supported Platforms

Platform	Method	What It Does
LinkedIn	Playwright	Login → Easy Apply → fill multi-step form → upload resume → submit
Indeed	Playwright	Login → click Apply → fill form → submit
Glassdoor	Playwright	Login → click Apply → fill form → submit
External sites	Playwright	Navigate to URL → detect forms → fill and submit
Craigslist	Direct contact	Extract email/phone from post → send email directly

How Playwright Works

Playwright is a browser automation library. It controls a real Chromium browser (headless — no visible window):

1. Launch headless Chromium
2. Navigate to the login page
3. Fill email + password fields
4. Click submit

5. Wait for page to load
6. Navigate to the job posting
7. Click "Apply" / "Easy Apply"
8. Fill form fields (name, email, phone, location, resume upload)
9. Submit the application
10. Close the browser

Craigslist Special Handling

Craigslist jobs don't have application portals. The post itself contains contact info:

- **Email found** → send personalized email with resume attached
- **Phone found** → log for manual follow-up (can't auto-call)
- **"Text" mentioned** → log for SMS follow-up
- **Nothing found** → log for manual review

The Craigslist email style is more casual than formal:

```
Hi,  
I saw your Craigslist post for {title} and wanted to reach out.  
I have experience in project management and would love to discuss this.  
My resume is attached. Feel free to call or text me anytime.  
Best, {name}
```

Anti-Bot Handling

- **LinkedIn CAPTCHA/security challenge** → logs as `blocked`, falls back to email outreach
- **2FA required** → logs as `blocked`, user needs to login manually once
- **Form too complex** (>5 steps) → logs as `partial`
- **Page structure changed** → form fill fails gracefully, logs error

Troubleshooting

Problem	Cause	Fix
---------	-------	-----

LinkedIn apply blocked	CAPTCHA or security challenge	User logs in manually once, passes verification
Indeed apply fails	Page structure changed	Update selectors in applier.py
Form fields not filled	CSS selectors don't match	Update the selector patterns
Craigslist email not sent	SMTP credentials wrong	Check email settings in dashboard
Playwright not installed	Missing dependency	<code>pip install playwright</code> && <code>playwright install chromium</code>
Browser crashes	Memory issue on server	Reduce concurrent applications

Feature 5: Email Outreach

What It Does

Sends a personalized email to the hiring manager for every job, regardless of whether the platform apply succeeded. This is a second channel — maximum coverage.

Email Structure

```
To: {hiring_manager_email}
From: {candidate_name} <{candidate_email}>
Subject: Application: {job_title} at {company} – {candidate_name}

Dear {contact_name},

{tailored_cover_letter}
```

{signature}

Resume attached

LinkedIn: {linkedin_url}

Job posting: {job_url}

Gmail SMTP Setup

The user needs a Gmail App Password (not their regular password):

1. Go to Google Account → Security
2. Enable 2-Step Verification
3. Go to App Passwords
4. Generate a password for "Mail"
5. Use that 16-character password in settings

Troubleshooting

Problem	Cause	Fix
Authentication failed	Wrong App Password	Regenerate at Google Account → Security → App Passwords
Emails not sending	Less secure apps disabled	Use App Password (not regular password)
Emails going to spam	New sender reputation	First few emails may land in spam — improves over time
Attachment too large	Resume file >25MB	Keep resume under 10MB
Rate limited	>500 emails/day	We send max 10/day — shouldn't happen

Feature 6: Web Dashboard

What It Does

Single-page web application served by the FastAPI backend. No build step, no npm, no framework.

Tabs

1. **Dashboard** — Stats (jobs scraped, sent, failed, pending), schedule info, recent applications
2. **Jobs** — Scraped jobs table with "Scrape Now" button
3. **Applications** — Full history with status badges
4. **Settings** — All configuration (candidate profile, search, platforms, resume mode, credentials, LLM, email)

Design System

Linear-inspired dark dashboard:

- Background: `#08090a` (near-black)
- Accent: `#5e6ad2` (indigo)
- Font: Inter (Google Fonts)
- Borders: semi-transparent white `rgba(255, 255, 255, 0.08)`
- No gradients, no glassmorphism, no emoji

API Communication

All calls go to the same origin (the Railway URL). The frontend is served at `/app` and the API at `/api/*`.

Troubleshooting

Problem	Cause	Fix
Dashboard not loading	Frontend files not served	Check main.py mounts static files correctly

Settings not saving	API error	Check browser console for error messages
Stats show 0	No pipeline run yet	Click "Scrape Now" or "Run Now"
Schedule not updating	Scheduler not running	Check main.py lifespan function

Part 3: Database Reference

Tables

settings

Key-value store. All configuration lives here.

Key	Shape	Example
candidate	<code>{name, email, phone, linkedin, location, title_target}</code>	<code>{"name": "John Smith", "email": "john@gmail.com"}</code>
search	<code>{job_titles[], locations[], platforms[], region, jobs_per_day, hours_old, min_salary}</code>	<code>{"job_titles": ["Project Manager"], "region": "us"}</code>
llm	<code>{api_base, api_key, model, max_tokens, temperature}</code>	<code>{"model": "openai/gpt-4o-mini"}</code>

platform_credentials	{linkedin: {email, password}, indeed: {email, password}}	{"linkedin": {"email": "john@gmail.com"}}
email	{smtp_host, smtp_port, username, password, from_name, from_email, signature}	{"smtp_host": "smtp.gmail.com"}
resume_mode	"honest" or "enhanced"	"honest"
base_resume	Full resume text (markdown)	"# John Smith\n..."
base_cover_letter	Cover letter template (markdown)	"Dear Hiring Manager, ..."
schedule_hour	int (0-23)	8
schedule_minute	int (0-59)	0

jobs

Scraped job postings.

Column	Type	Notes
id	INTEGER	Auto-increment primary key
title	TEXT	Job title
company	TEXT	Company name
location	TEXT	Job location

description	TEXT	Full job description (up to 3000 chars)
job_url	TEXT	Link to the original posting
date_posted	TEXT	When the job was posted
site	TEXT	Which platform (linkedin, indeed, etc.)
relevance_score	REAL	0-100 score from scoring algorithm
status	TEXT	new → applied or skipped

applications

Sent applications — the permanent record.

Column	Type	Notes
id	INTEGER	Auto-increment primary key
job_id	INTEGER	Foreign key to jobs table
title	TEXT	Job title
company	TEXT	Company name
contact_name	TEXT	Who we contacted
contact_email	TEXT	Email we sent to
contact_source	TEXT	How we found the contact

<code>resume_text</code>	TEXT	The tailored resume that was sent
<code>cover_letter</code>	TEXT	The tailored cover letter that was sent
<code>resume_mode</code>	TEXT	<code>honest</code> or <code>enhanced</code>
<code>apply_method</code>	TEXT	<code>linkedin_easy_apply</code> , <code>craigslist_email</code> , etc.
<code>apply_status</code>	TEXT	<code>applied</code> , <code>blocked</code> , <code>failed</code> , <code>skipped</code>
<code>outreach_status</code>	TEXT	<code>sent</code> , <code>failed</code> , <code>skipped</code>
<code>error</code>	TEXT	Error message if something failed

Part 4: API Reference

All endpoints return JSON. Base URL is the Railway deployment URL.

Method	Path	Purpose	Response
GET	<code>/api/health</code>	Health check	<code>{status: "ok", time: "..."} </code>
GET	<code>/api/settings</code>	Get all settings	Full settings (secrets masked)

PUT	/api/settings	Update settings	{status: "ok", updated: [...]}
GET	/api/jobs?limit=50	List scraped jobs	Array of job objects
GET	/api/applications?limit=100	Application history	Array of application objects
GET	/api/applications/stats	Dashboard stats	{total_jobs_scraped, sent, failed, pending}
GET	/api/platforms	Available platforms	Array of {id, name, type}
POST	/api/pipeline/run	Run full pipeline	{status: "completed"}
POST	/api/pipeline/scrape	Just scrape jobs	{scraped: N, new: N}
GET	/api/schedule	Get schedule	{enabled, next_run, hour, minute}
PUT	/api/schedule?hour=3&minute=0	Update Schedule	{status: "ok"}

Part 5: Troubleshooting Quick Reference

System Won't Start

Symptom	Check	Fix
---------	-------	-----

ModuleNotFoundError	requirements.txt	<code>pip install -r requirements.txt</code>
playwright not installed	Playwright	<code>pip install playwright</code> && <code>playwright install chromium</code>
Address already in use	Port conflict	Kill other process or change port
Database locked	SQLite	Only one process can write at a time

Pipeline Fails

Step	Symptom	Fix
Scrape	No jobs returned	Check search terms, locations, region
Select	All jobs skipped	Check if already applied (applications table)
Enrich	LLM returns None	Check API key, model name, rate limits
Contact	All fallback	Normal for some companies — fallback email used
Apply	Blocked by platform	User needs to login manually once
Send	SMTP auth failed	Check Gmail App Password

Dashboard Issues

Symptom	Fix
Blank page	Check browser console for JS errors
Settings not loading	Check <code>/api/settings</code> returns data
Stats not updating	Click "Run Now" or wait for scheduled run
Schedule not working	Check APScheduler is started in main.py lifespan

Common User Errors

Mistake	Consequence	Prevention
Regular Gmail password	SMTP auth fails	Use App Password (16-char)
Wrong API key	LLM enrichment fails	Test with setup wizard
No platforms selected	Nothing to scrape	Check at least one platform
No job titles	Nothing to scrape	Add at least one title
Enhanced mode + bad LLM	Fabricated resume	User should review enhanced resumes

Part 6: Suggestions & Improvements

Short-Term (Easy Wins)

Add a "Test Connection" button in settings — verify LLM API, SMTP, and platform credentials before running the pipeline.

Add a "Preview" feature — show the tailored resume + cover letter before sending. User can approve or reject.

Add daily email summary — send the user a summary of what was applied to each day (jobs, companies, statuses).

Add job blacklist — let the user exclude specific companies or job titles they don't want.

Add salary filter display — show the salary range on each job in the dashboard so the user can see what they're applying to.

Medium-Term (More Work)

Add a "dry run" mode — scrape, enrich, and show what WOULD be applied without actually sending anything. Good for testing.

Add resume PDF generation — convert the markdown resume to a clean PDF before attaching. Looks more professional.

Add analytics — track response rates per platform, per resume mode, per style variant. See which combinations get the most callbacks.

Add job alerts — notify the user when a high-score job is found (e.g., >90 relevance score).

Support multiple resumes — let the user upload different base resumes for different role types (PM vs. Scrum Master vs. Program Manager).

Long-Term (Significant)

Add interview prep — when a callback comes in, generate interview prep materials based on the job description and the resume that was sent.

Add follow-up automation — if no response after 3 days, send a follow-up email.

Add LinkedIn Easy Apply profile optimization — suggest profile changes based on the jobs being applied to.

Add cover letter A/B testing — track which cover letter styles get the most responses and optimize over time.

Multi-user support — let multiple job seekers use the same deployment with separate accounts.

Part 7: Security Considerations

What's Stored

Data	Where	Risk Level
Platform credentials	SQLite (settings table)	Medium — encrypted at rest by Railway
Gmail App Password	SQLite (settings table)	Medium — same as above
LLM API key	SQLite (settings table)	Low — can be regenerated
Resume content	SQLite (settings table)	Low — not sensitive
Application history	SQLite (applications table)	Low — public job data

Recommendations

1. **Use a dedicated Gmail account** for the automation — not your primary email.
 2. **Use App Passwords** — never store your main Google password.
 3. **Enable Railway's environment variables** for secrets instead of storing in SQLite.
 4. **Regularly rotate API keys** — especially if sharing the deployment.
 5. **Review enhanced resumes** before they're sent — the AI may embellish too much.
-

Part 8: File Reference

Complete File Structure

```
job-hunter/
├── docs/
│   ├── 00-OVERVIEW.md           Project overview
│   ├── 01-ARCHITECTURE.md       Technical architecture
│   ├── 02-PIPELINE.md           7-step daily flow
│   ├── 03-BACKEND.md            Backend implementation guide
│   ├── 04-FRONTEND.md           Frontend design guide
│   ├── 05-DEPLOYMENT.md         Deployment instructions
│   ├── 06-SKILLS.md             Compiled skill references
│   ├── 07-TASKS.md              Implementation task list
│   └── FRONTEND-DESIGN.md        Complete design system
├── backend/
│   ├── main.py                  FastAPI app, routes, scheduler
│   ├── database.py              SQLite CRUD operations
│   ├── requirements.txt          Python dependencies
│   ├── Procfile                 Railway deployment command
│   ├── railway.json             Railway configuration
│   └── pipeline/
│       ├── __init__.py          Package marker
│       ├── scrape.py            Multi-platform job scraping
│       ├── enrich.py            Resume/cover letter enrichment
│       ├── contacts.py          Free hiring manager lookup
│       ├── applier.py           Platform apply + Craigslist direct
│       └── send.py              Email outreach via Gmail SMTP
├── frontend/
│   └── index.html               Single-file dashboard
└── AGENTS.md                   Anti-Gravity IDE instructions
```

Dependencies

```
fastapi>=0.104.0    Web framework
uvicorn>=0.24.0      ASGI server
apscheduler>=3.10.0  Background scheduler
```

python-jobsby>=1.1.0	Multi-site job scraper
pyyaml>=6.0	YAML parsing
requests>=2.28.0	HTTP client
dnspython>=2.4.0	DNS lookups (for email verification)
python-multipart>=0.0.6	Form data parsing
playwright>=1.40.0	Browser automation

End of report. For questions or issues, refer to the specific section above or check the individual docs/ files for implementation details.